

ROM HACKING GUIDE

HOW TO IMPLEMENT A SPLASH SCREEN IN SONIC 1 (GITHUB DISASSEMBLY) - BY THEGAMER2000

-TABLE OF CONTENTS-

Each sections' title is color-coded for easy referencing.

INTRODUCTION - PAGE 2

SPLASH SCREEN GRAPHIC CREATION - PAGE 3

SPLASH SCREEN GRAPHIC - USING RETRO GRAPHICS TOOLKIT - PAGE 4

SPLASH SCREEN GRAPHIC - USING SONPLN - PAGE 14

SPLASH SCREEN GRAPHIC - TINKERING WITH THE ASM FILES - PAGE 18

BUILDING THE FINAL BUILD: SPLASH SCREEN IMPLEMENTED? - PAGE 34

DOCUMENT INFO, CREDITS, & NOTES - PAGE 36



-INTRODUCTION-

So you want to have your own custom splash screen in your Sonic 1-based ROM hack? Well, you're in luck! This guide will teach you how to do just that. **Keep in mind that this guide only applies to the GitHub-based Sonic 1 disassembly.**

Before we get started, **make sure that you have the applications listed below on your computer** (links are provided if you don't have one or more of these applications):

[7-Zip](#)

ComputerNerd's [Retro Graphics Toolkit](#)

MainMemory's [SonPLN](#)

SonicRetro's [GitHub disassembly of Sonic 1 \(Not required if you already have your disassembly on hand, make sure that it's GitHub-based\)](#)

Once you finish extracting the **.7Z file** (RGTK) and/or the **.ZIP file** (S1GHDISA, SPLN, 7ZIP), it's time to begin the implementation process of a custom static splash screen!

-SPLASH SCREEN GRAPHIC CREATION-

Of course, before you actually start implementing your custom splash screen, you must create your splash screen graphic! **For the graphic to appear correctly in your hack, make sure to set your image to 320x224 pixels.**

Provided below is a sample graphic that you can use to follow along in this guide.

[Tutorial Graphic](#)



Sample graphic. Click on the link above to download this graphic. (PNG)

Once you've created your graphic or downloaded the sample graphic above, you may proceed on to the next section, **Splash Screen Graphic - Using the Retro Graphics Toolkit.**

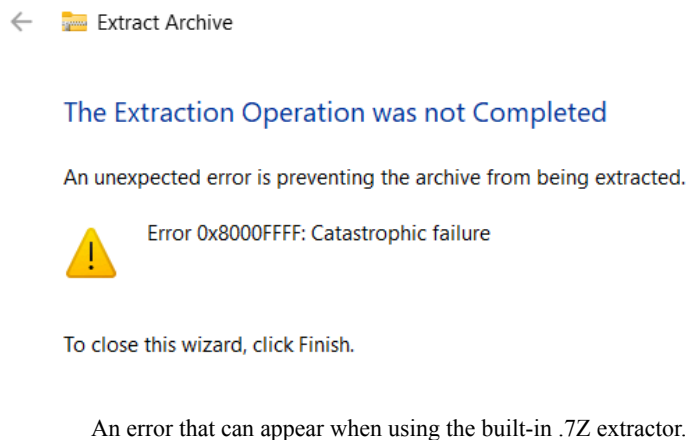
-SPLASH SCREEN GRAPHIC - USING RETRO GRAPHICS TOOLKIT-

Before you use the Retro Graphics Toolkit application, you will need to extract the .7Z file with 7-Zip. If you don't have 7-Zip already installed on your computer, you may do so.

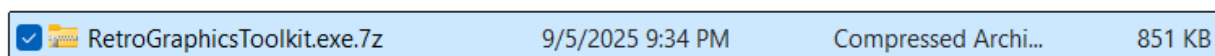
Once you have 7-Zip installed, download the .7Z file containing the Retro Graphics Toolkit application.

Now that you have 7-Zip and the RGTK .7Z file on your computer, you can extract the .7Z file in the directory of your choice.

WARNING: there may be a chance that extracting the .7Z file using Windows 11's built-in .7Z extractor will result in an error message like the one below:

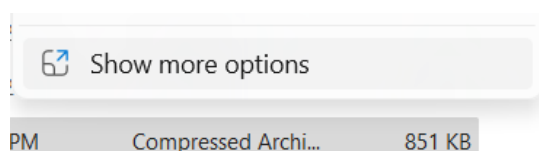


Find the .7Z file and right-click on it.



Find the .7Z file and right-click on it.

If using Windows 11, go to “Show more options” and then “7-Zip.” If you’re using an older version of Windows, just right-click on the .7Z file and then go to “7-Zip.”

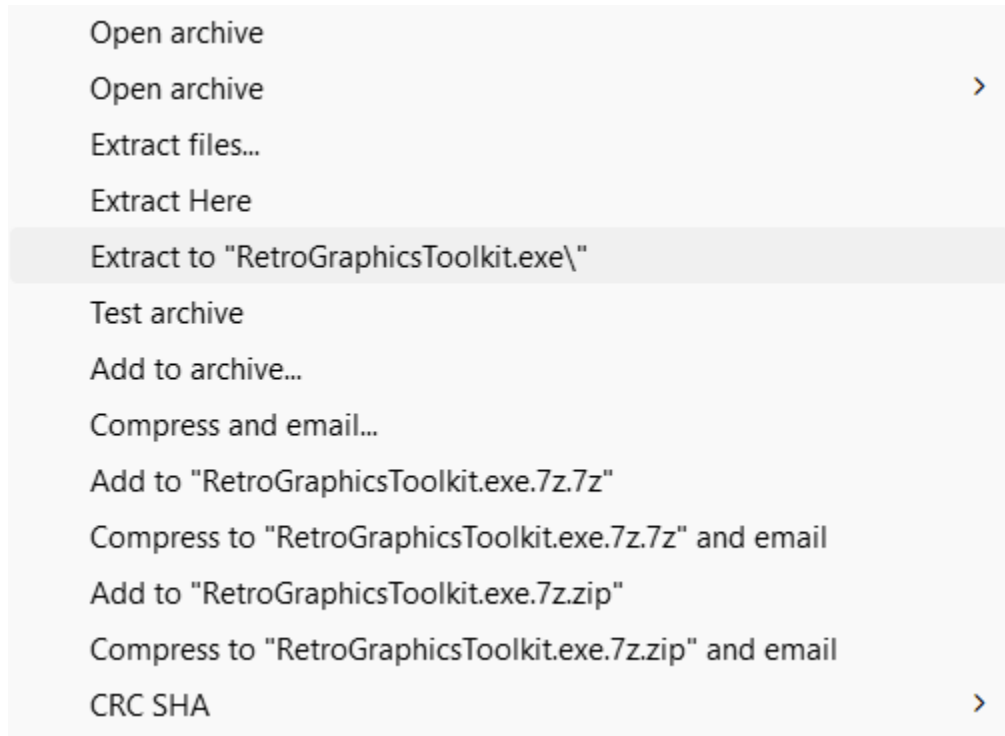


Click on “Show more options” (Windows 11 only).



Go to “7-Zip.”

In the drop-down menu, click on “Extract to “RetroGraphicsToolkit.exe\.”



Click on “Extract to “RetroGraphicsToolkit.exe\.

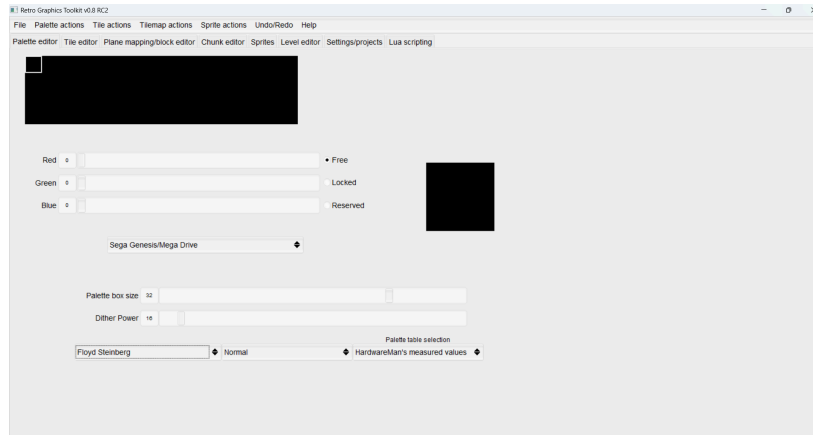
If done correctly, the **file** should be **extracted** and the **RGTK folder** will be **created**.



Now that the **Retro Graphics Toolkit** application is installed, you may now begin the **first** step in the **implementation process** below!

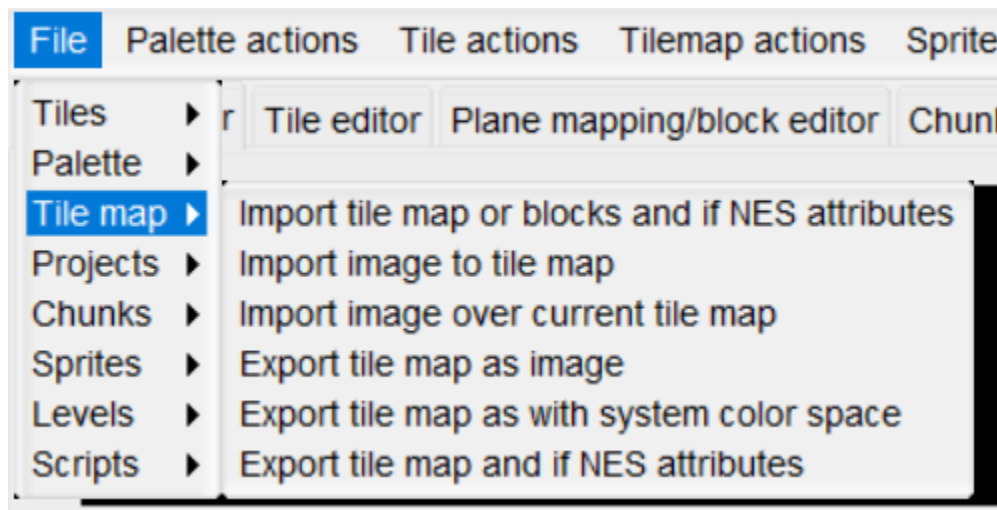
The **first** step in the implementation process involves the creation of a **tilemap file**, a **palette file**, and a **file containing the graphics’ art tiles**. The **Retro Graphics Toolkit** application will assist in the creation of the files listed above.

Open up the **Retro Graphics Toolkit** application. You should be greeted with a window that looks like or looks similar to the screenshot below:

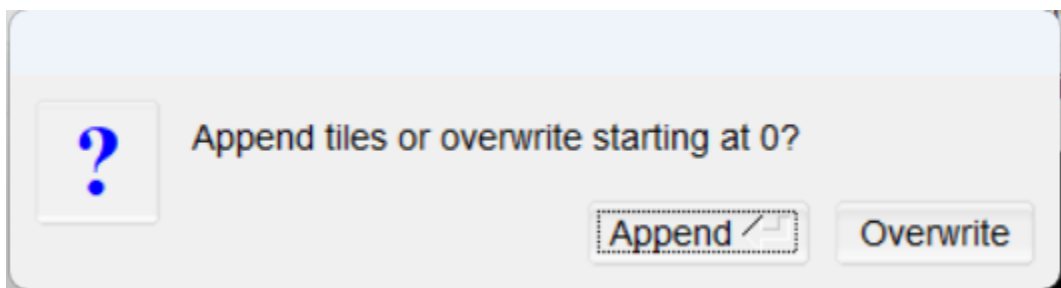


Open up the Retro Graphics Toolkit application.

To open your graphic into this application, go to **File -> Tile map -> Import image to tile map**. The application will ask you if you'd like to append or overwrite tiles. Click on **Overwrite**.

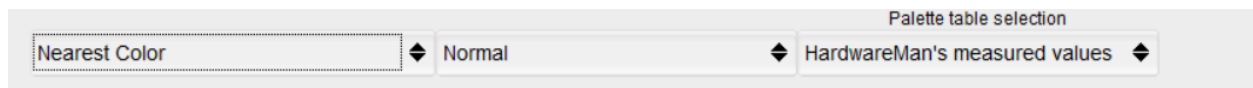


Go to File -> Tile map -> Import image to tile map.



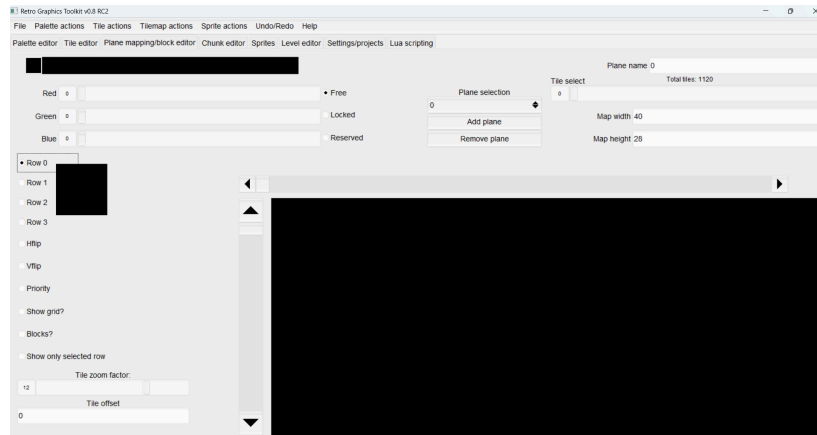
Click on "Overwrite."

Now that the graphic is open within the application, you can change the **dithering option** from **"Floyd Steinburg"** to **"Nearest Color."** This is just a recommendation, but if you'd like to have **solid colors** on your splash screen graphic, you must select this option.

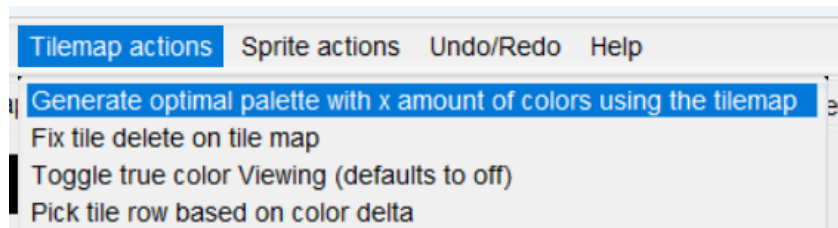


Set the left drop-down box to “Nearest Color” if you wish to have solid colors on your splash screen graphic.

Go to the **Plane mapping/block editor**. While on this screen, go to **Tilemap actions -> Generate optimal palette with x amount of colors using the tilemap**.

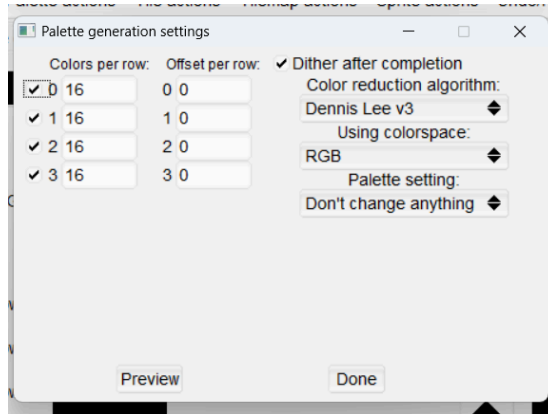


Go to the Plane mapping/block editor.



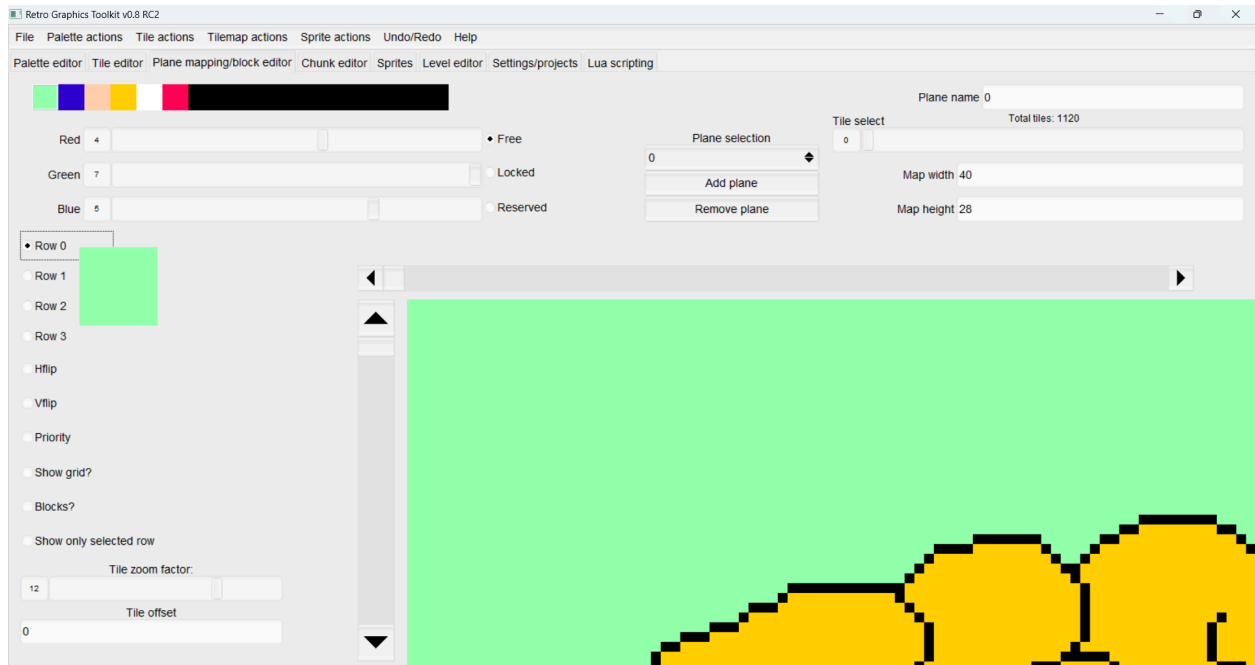
Go to Tilemap actions -> Generate optimal palette with x amount of colors using the tilemap.

Click on “**Preview**” and then “**Done.**”



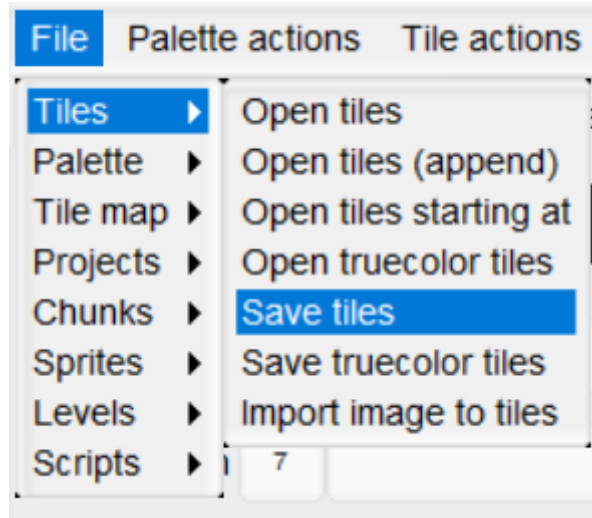
Click on “Preview” and then “Done.”

If done correctly, your graphic should appear. If the map width is **40** and the map height is **28**, you should be good to go and can continue on to the next step.

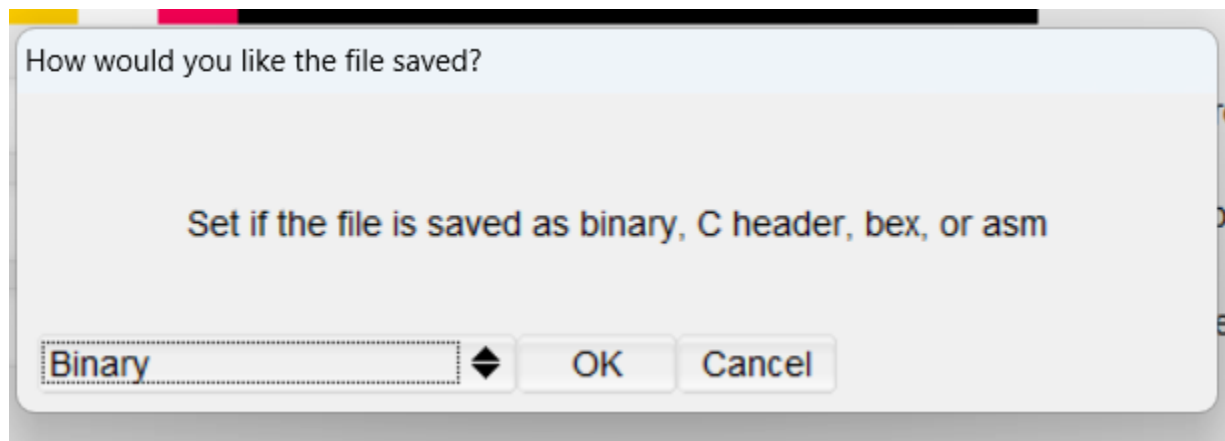


If done correctly, your graphic should appear.

We'll save the **tiles** first. Go to **File -> Tiles -> Save tiles**. Set the file type to “**Binary**” and click “**OK.**” A pop-up window will appear.

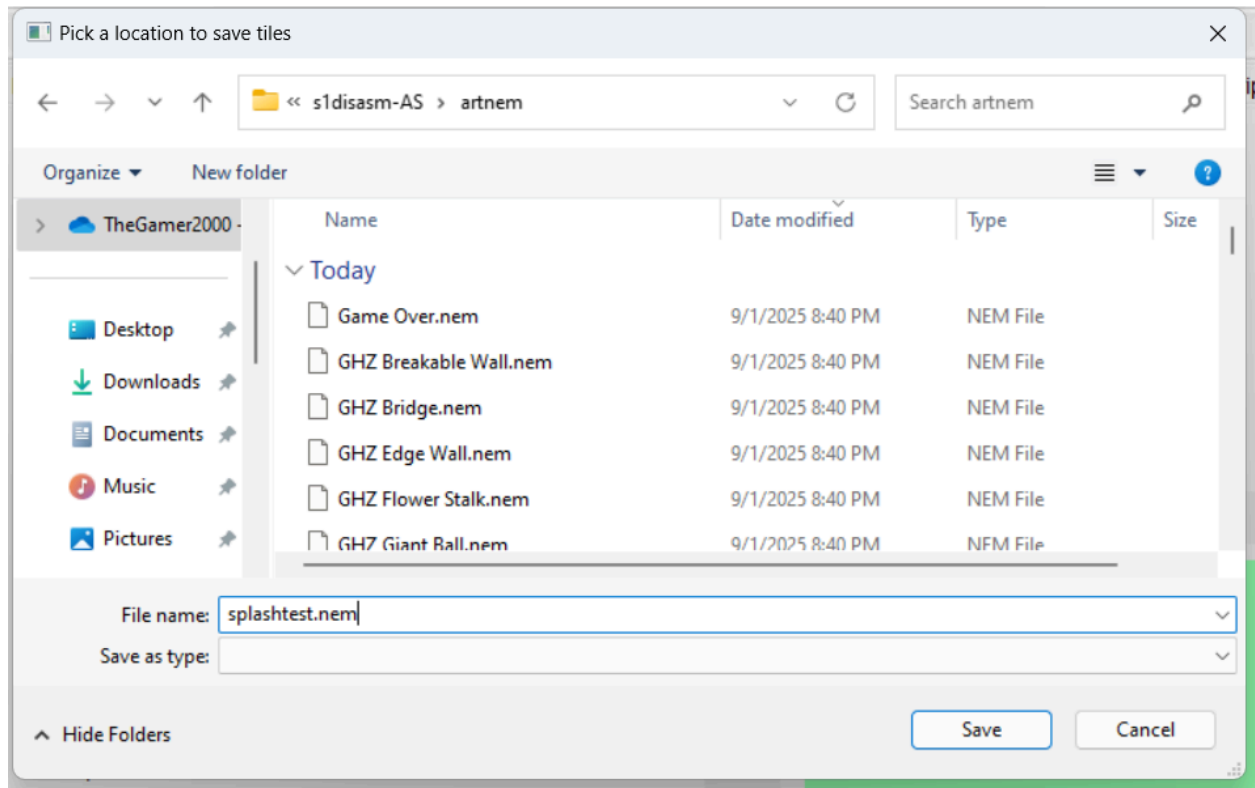


Go to File -> Tiles -> Save tiles.

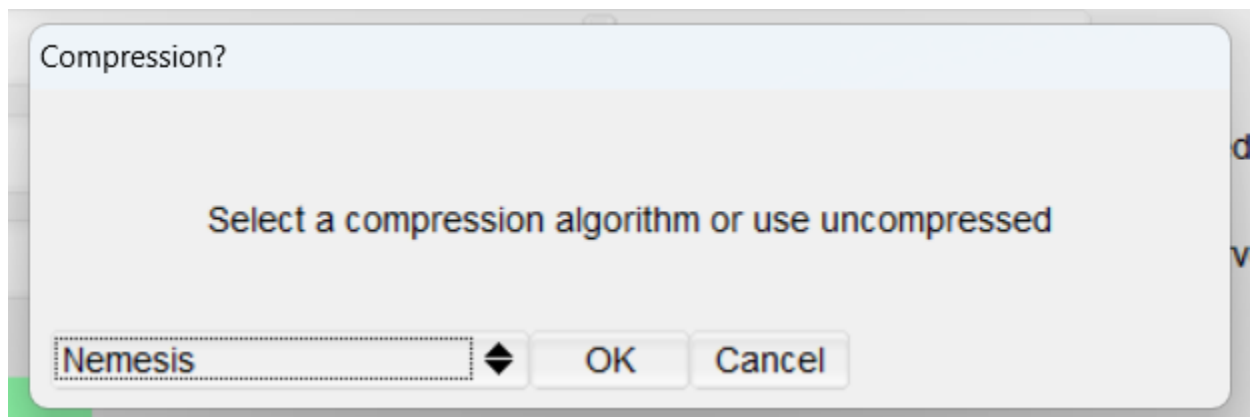


Set the file type to "Binary" and click "OK."

Save your **tiles** file in the **artnem** folder in your **disassembly**. You may name it whatever you'd like, but to keep things simple, we'll use **splashtest.nem** as the file name. Click "**Save.**"

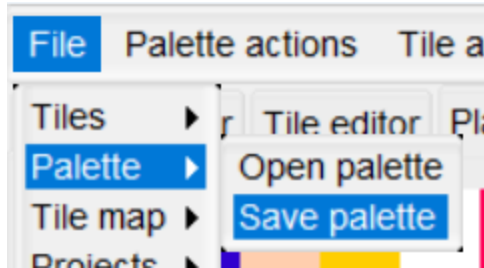


Set the compression algorithm to “Nemesis” and click “OK.”

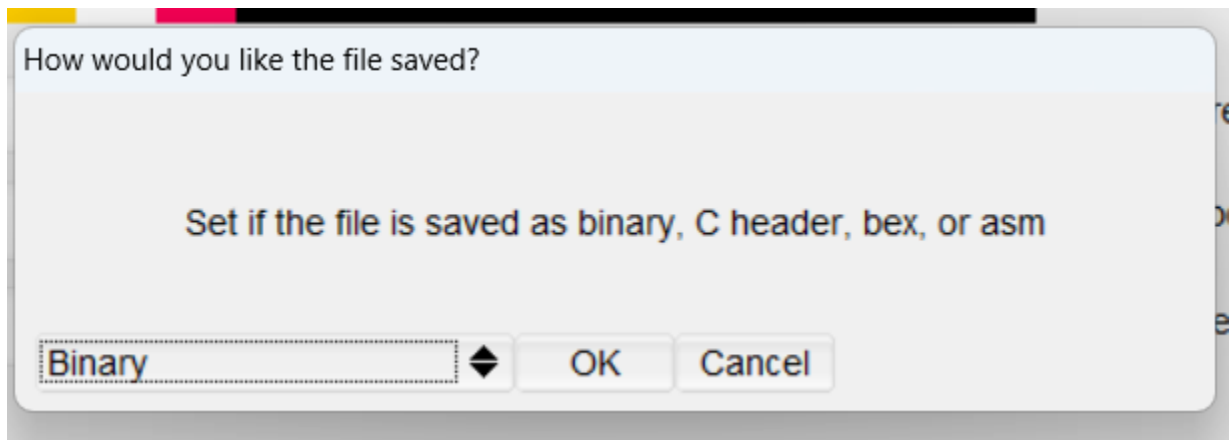


Select “Nemesis” and then “OK.”

Now, we’ll save the **palette**. Go to **File -> Palette -> Save palette**. Click “OK” twice. Set the file type to “Binary” and click “OK.” A pop-up window will then appear.

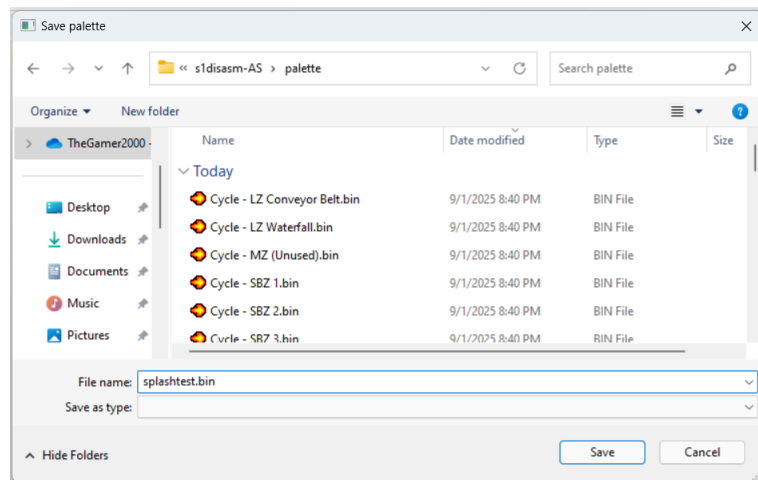


Go to File -> Palette -> Save palette. Click “OK” twice.

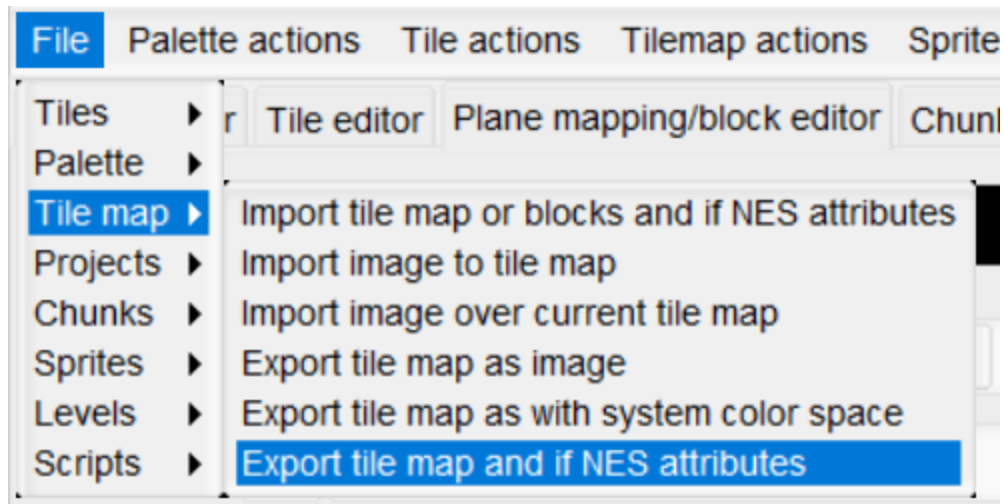


Set the file type to “Binary” and click “OK.”

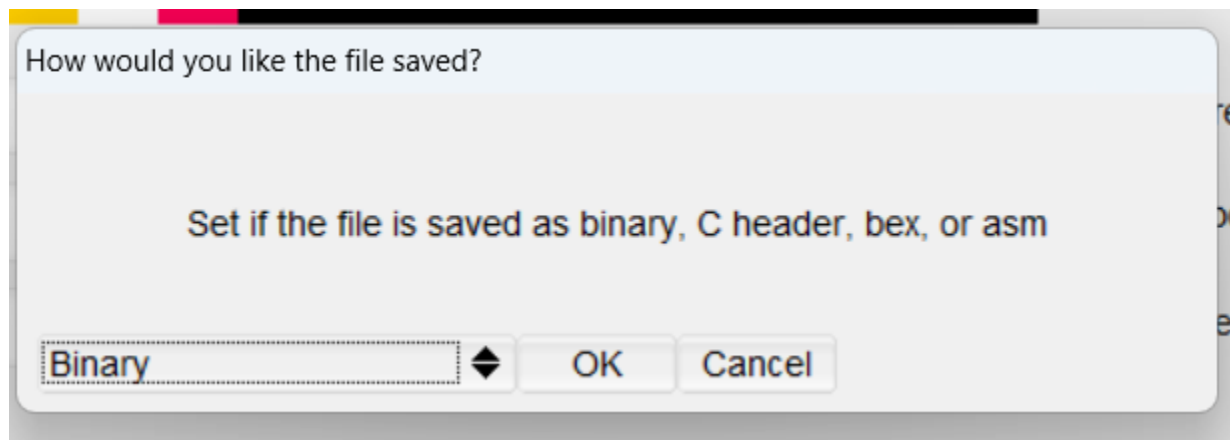
Save your **palette** file in the **palette** folder in your **disassembly**. You may name it whatever you’d like, but to keep things simple, we’ll use **splashtest.bin** as the file name. Click “**Save.**”



Finally, we’ll save the **tilemap** file. Go to **File -> Tile map -> Export tile map and if NES attributes**. Set the file type to “**Binary**” and click “**OK.**” A pop-up window will appear.

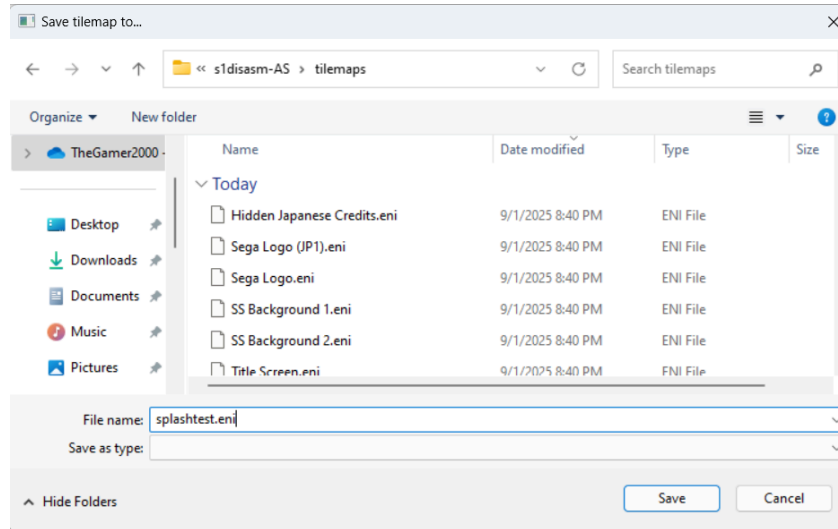


Go to File -> Tile map -> Export tile map and if NES attributes.

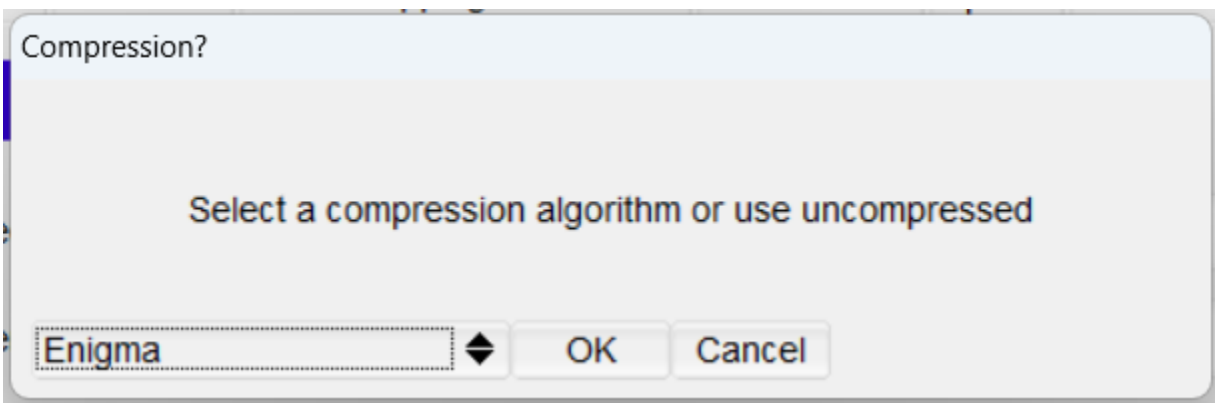


Set the file type to "Binary" and click "OK."

Save your **tilemap** file in the **tilemaps** folder in your **disassembly**. You may name it whatever you'd like, but to keep things simple, we'll use **splashtest.eni** as the file name. Click **"Save."**



Set the compression algorithm to “**Enigma**” and click “**OK.**”



Select “Enigma” and then “OK.”

Now that all of the required files have been created and placed in the correct folders, you may now proceed to the next section, **Splash Screen Graphic - Using SonPLN.**

-SPLASH SCREEN GRAPHIC - USING SONPLN-

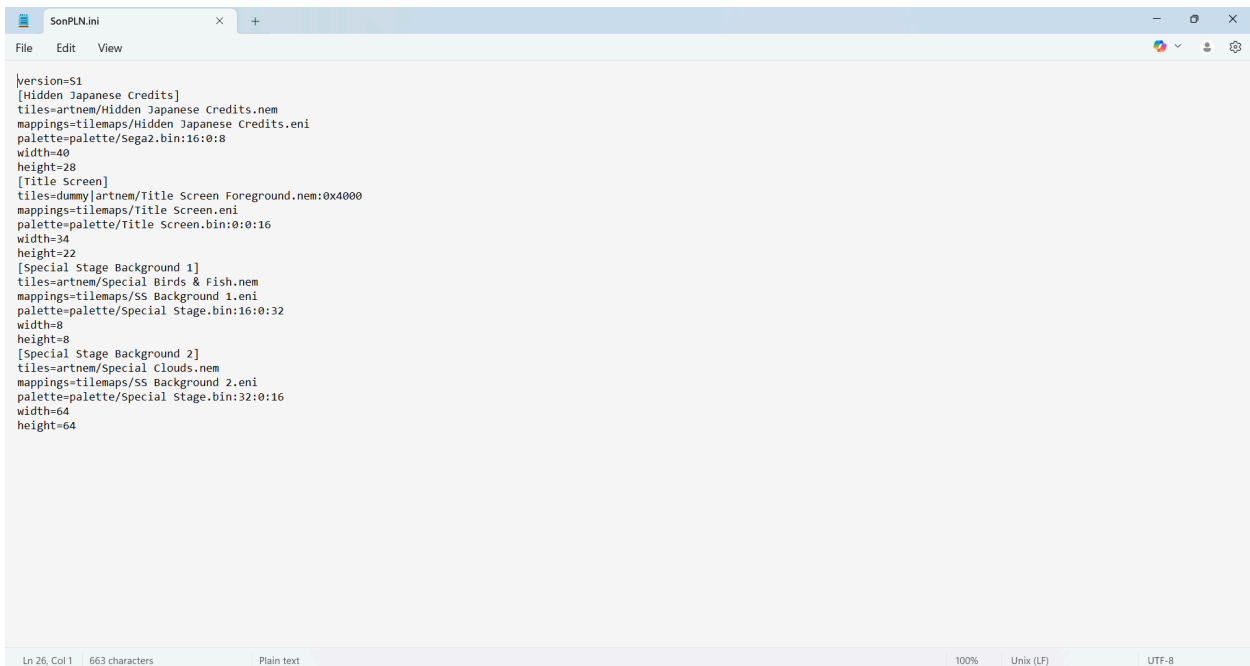
The **second** step in the implementation process involves the usage of **SonPLN**. The **GitHub Disassembly** should contain a **SonPLN.ini** file preloaded.

First, open **SonPLN.ini** in **Notepad**.

	sonic1.flex.json	9/1/2025 8:40 PM	JSON File	101 KB
	SonPLN.ini	9/1/2025 8:40 PM	INI File	1 KB
	Variables.asm	9/1/2025 8:40 PM	ASM File	24 KB

Open SonPLN.ini in Notepad.

You should see this upon opening the file.



```
version=S1
[Hidden Japanese Credits]
tiles=artnem/Hidden Japanese Credits.nem
mappings=tilemaps/Hidden Japanese Credits.eni
palette=palette/Sega2.bin:16:0:8
width=40
height=28
[Title Screen]
tiles=dummy/artnem/Title Screen Foreground.nem:0x4000
mappings=tilemaps/Title Screen.eni
palette=palette/Title Screen.bin:0:0:16
width=34
height=22
[Special Stage Background 1]
tiles=artnem/Special Birds & Fish.nem
mappings=tilemaps/SS Background 1.eni
palette=palette/Special Stage.bin:16:0:32
width=8
height=8
[Special Stage Background 2]
tiles=artnem/Special Clouds.nem
mappings=tilemaps/SS Background 2.eni
palette=palette/Special Stage.bin:32:0:16
width=64
height=64
```

At the **bottom** of the list, press the **ENTER** key to go down by one space and **copy the text below** into **Notepad**:

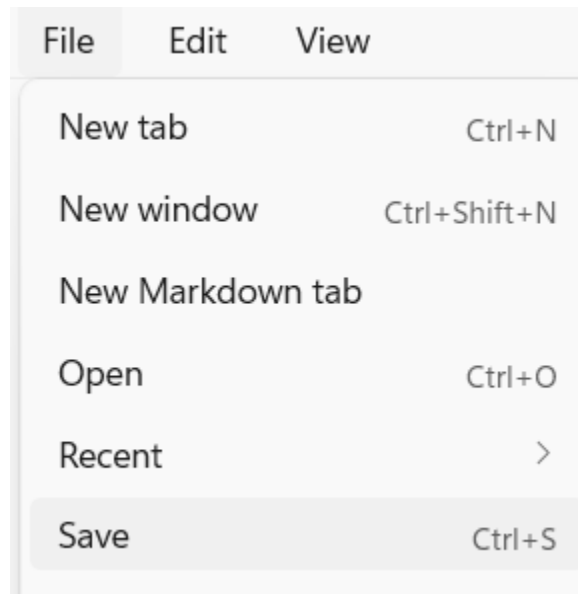
```
[SplashScreenTest]
tiles=artnem/splashtest.nem
mappings=tilemaps/splashtest.eni
palette=palette/splashtest.bin:0:0:16
width=40
height=28
```

Copy the text above. You can change [SplashScreenTest] and “splashtest” in the text to the name of the files you saved (if you created your own file names).

```
[Special Stage Background 2]
tiles=artnem/Special Clouds.nem
mappings=tilemaps/SS Background 2.eni
palette=palette/Special Stage.bin:32:0:16
width=64
height=64
[SplashScreenTest]
tiles=artnem/splashtest.nem
mappings=tilemaps/splashtest.eni
palette=palette/splashtest.bin:0:0:16
width=40
height=28
```

Paste the text above in Notepad.

Once completed, go to **File -> Save. Close Notepad.**



Go to File -> Save and then close Notepad.

Now, open **SonPLN.ini** in **SonPLN**.

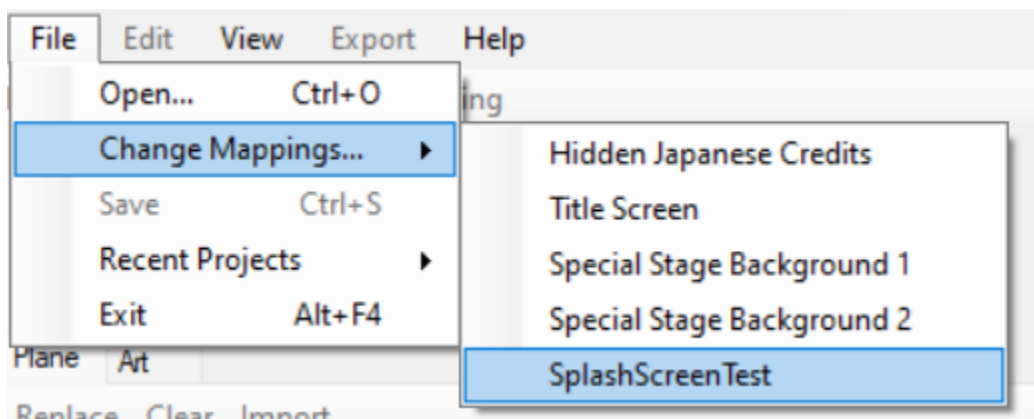
	sonic1.flex.json	9/1/2025 8:40 PM	JSON File	101 KB
	SonPLN.ini	9/1/2025 8:40 PM	INI File	1 KB
	Variables.asm	9/1/2025 8:40 PM	ASM File	24 KB

Open SonPLN.ini in SonPLN.

You should see this upon opening the file.

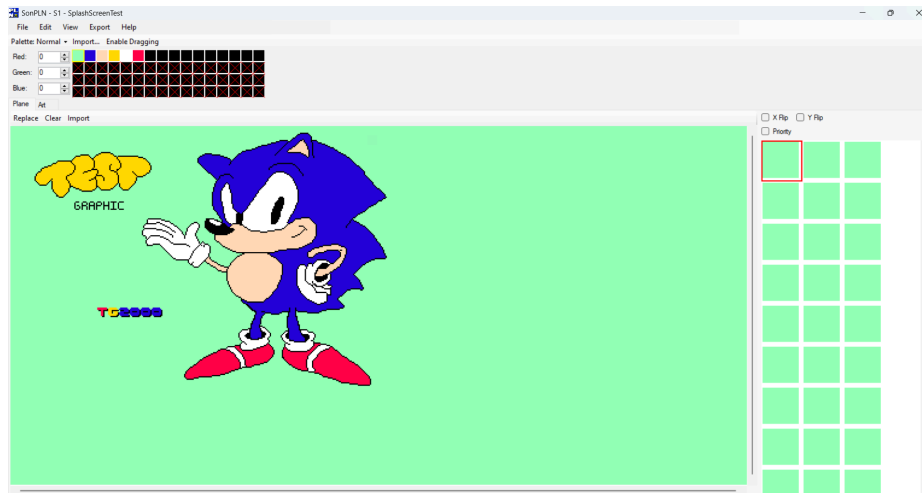


Go to **File -> Change Mappings... -> SplashScreenTest** (Or the name you gave the mapping in Notepad).



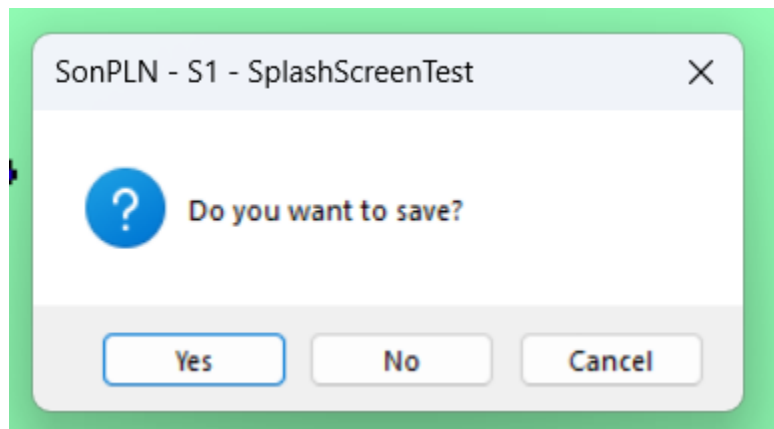
Go to **File -> Change Mappings... -> SplashScreenTest** (Or the name you gave the mapping in Notepad).

If done correctly, you should see this (or the graphic you created):



The test graphic (or your graphic) should appear on the plane.

Close SonPLN. Click “Yes” on the **Save** prompt.



Click “Yes” on the Save prompt.

The preceding task was done to make sure that all of the files required to make the splash screen work are functional. You may now proceed to the hardest section, **Splash Screen Graphic - Tinkering With the ASM Files.**

-SPLASH SCREEN GRAPHIC - TINKERING WITH THE ASM FILES-

The **third and final** step in the implementation process involves the modification of **various ASM files** found within the **GitHub Disassembly**. This step is tedious, so read carefully.

WARNING: Before you modify the ASM files, make a backup copy of your disassembly in case you mess up. Run build.bat and open up the created build to make sure that your hack works as well.



s1disasm-AS
(Tutorial) -
BACKUP

Make a backup copy of your disassembly.

	.gitignore	9/1/2025 8:40 PM	GITIGNORE File	1 KB
☒ 	build.bat	9/1/2025 8:40 PM	Windows Batch File	1 KB
	build.lua	9/1/2025 8:40 PM	LUA File	2 KB
Run build.bat, this batch file should be found in the root of your disassembly.				
	sonic.lst	9/2/2025 12:49 PM	LST File	7,971 KB
☒ 	s1built.bin	9/2/2025 12:49 PM	BIN File	512 KB

Open the generated build in a Sega Genesis/Mega Drive emulator.



If the build works, you may continue on to the ASM editing portion. (Note that your build may look different.)

First, open **sonic.asm** in **Notepad**. It should be found in the **root** directory of your **disassembly**.

	S1SSEdit.ini	9/1/2025 8:40 PM	INI File	1 KB
☒ 	sonic.asm	9/1/2025 8:40 PM	ASM File	223 KB
	sonic1.flex.json	9/1/2025 8:40 PM	JSON File	101 KB

Open sonic.asm in Notepad.

You should see this upon opening the file:

```

sonic.asm
File Edit View

; =====
; |          Sonic the Hedgehog Disassembly for Sega Mega Drive          |
; =====
;
; Disassembly created by Hivebrain
; thanks to drx, Stealth and Esrael L.G. Neto
; =====

cpu 68000

EnableSRAM      = 0 ; change to 1 to enable SRAM
BackupSRAM      = 1
AddressSRAM     = 3 ; 0 = odd+even; 2 = even only; 3 = odd only

; change to 0 to build the original version of the game, dubbed REV00
; change to 1 to build the later version, dubbed REV01, which includes various bugfixes and enhancements
; change to 2 to build the version from Sonic Mega collection, dubbed REVXB, which fixes the infamous "spike bug"
Revision        = 1

ZoneCount       = 6 ; discrete zones are: GHZ, MZ, SVZ, LZ, SLZ, and SBZ
FixBugs         = 0 ; change to 1 to enable bugfixes
zeroOffsetOptimization = 0 ; if 1, makes a handful of zero-offset instructions smaller

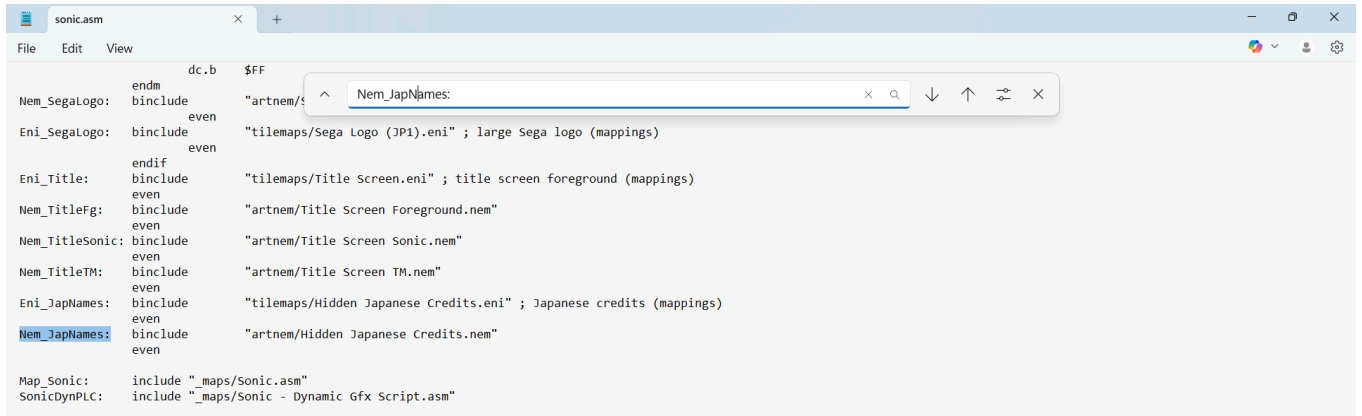
include "MacroSetup.asm"
include "Constants.asm"
include "Variables.asm"
include "Macros.asm"

; =====

StartOfRom:
Vectors:
    dc.l v_systemstack&$FFFFFF ; Initial stack pointer value
    dc.l EntryPoint            ; Start of program
    dc.l BusError              ; Bus error
    dc.l AddressError          ; Address error (4)
    dc.l IllegalInstr          ; Illegal instruction
    dc.l ZeroDivide            ; Division by zero

```

Locate **Nem_JapNames:** in the file (CTRL+F for faster searching).



Locate **Nem_JapNames:** in the .ASM file.

After “even,” press the **ENTER** key to go down by one space, then **copy the code below:**

```
Nem_SplashTest: bininclude      "artnem/splashtest.nem" ; Tiles
                    even
Eni_SplashTest:  bininclude      "tilemaps/splashtest.eni" ; Maps
                    even
```

Copy the text above. You can change “SplashTest” and “splashtest” in the text to the name of the files you saved (if you created your own file names).

```
Nem_JapNames:    bininclude      "artnem/Hidden Japanese Credits.nem"
                    even
Nem_SplashTest   bininclude      "artnem/splashtest.nem" ; Tiles
                    even
Eni_SplashTest   bininclude      "tilemaps/splashtest.eni" ; Maps
                    even|

Map_Sonic:       include "_maps/Sonic.asm"
SonicDynPLC:     include "_maps/Sonic - Dynamic Gfx Script.asm"
```

Paste the code above in Notepad.

Now, locate **Pal_Ending:** in the file (CTRL+F for faster searching).

```

; =====
; include "_inc/Palette Po"
; =====
; Palette data
; =====
binincludePalette macro path,{INTLABEL},{GLOBALSYMBOLS}
__LABEL__: bininclude path
__LABEL__end:
endm

Pal_SegaBG: binincludePalette "palette/Sega Background.bin"
Pal_Title: binincludePalette "palette/Title Screen.bin"
Pal_LevelSel: binincludePalette "palette/Level Select.bin"
Pal_Sonic: binincludePalette "palette/Sonic.bin"
Pal_GHZ: binincludePalette "palette/Green Hill Zone.bin"
Pal_LZ: binincludePalette "palette/Labyrinth Zone.bin"
Pal_LZWater: binincludePalette "palette/Labyrinth Zone Underwater.bin"
Pal_MZ: binincludePalette "palette/Marble Zone.bin"
Pal_SZ: binincludePalette "palette/Star Light Zone.bin"
Pal_SYZ: binincludePalette "palette/Spring Yard Zone.bin"
Pal_SBZ1: binincludePalette "palette/SBZ Act 1.bin"
Pal_SBZ2: binincludePalette "palette/SBZ Act 2.bin"
Pal_Special: binincludePalette "palette/Special Stage.bin"
Pal_SBZ3: binincludePalette "palette/SBZ Act 3.bin"
Pal_SBZ3Water: binincludePalette "palette/SBZ Act 3 Underwater.bin"
Pal_LZSonWater: binincludePalette "palette/Sonic - LZ Underwater.bin"
Pal_SBZ3SonWat: binincludePalette "palette/Sonic - SBZ3 Underwater.bin"
Pal_SSResult: binincludePalette "palette/Special Stage Results.bin"
Pal_Continue: binincludePalette "palette/Special Stage Continue Bonus.bin"
Pal_Ending: binincludePalette "palette/Ending.bin"

; =====
; Subroutine to wait for VBlank routines to complete
; =====

; ||| S U B R O U T I N E |||

WaitForVBlk:
Ln 2000, Col 12 11 of 227,972 characters Plain text 100% Unix (LF) UTF-8

```

Locate Pal_Ending: in the .ASM file.

After **Pal_Ending:** **binincludePalette** “palette/Ending.bin,” press the **ENTER** key to go down by one space, then **copy the code below:**

Pal_SplashTest: **binincludePalette** "palette/SplashTest.bin"

Copy the text above. You can change “SplashTest” in the text to the name of the files you saved (if you created your own file names).

```

Pal_SSResult: binincludePalette "palette/Special Stage Results.bin"
Pal_Continue: binincludePalette "palette/Special Stage Continue Bonus.bin"
Pal_Ending: binincludePalette "palette/Ending.bin"
Pal_SplashTest: binincludePalette "palette/SplashTest|.bin"

```

Paste the code above in Notepad.

Now, we’ll add a new **game mode** for the splash screen.

Locate **MainGameLoop:** in the file (**CTRL+F** for faster searching).

```

bsr.w  VDPSetupGame
bsr.w  DACDriverLoad
bsr.w  JoypadInit
move.b #id_Sega,(v_gamemode).w ; set game mode to Sega screen

MainGameLoop:
    move.b (v_gamemode).w,d0 ; load Game Mode
    andi.w #$1C,d0 ; limit Game Mode value to $1C max (change to a maximum of 7C to add more game modes)
    jsr   GameModeArray(pc,d0.w) ; jump to apt location in ROM
    bra.s MainGameLoop ; loop indefinitely
; =====
;
; Main game mode array
; =====

GameModeArray:
ptr_GM_Sega:  bra.w  GM_Sega      ; Sega Screen ($00)
ptr_GM_Title: bra.w  GM_Title     ; Title Screen ($04)
ptr_GM_Demo:  bra.w  GM_Level     ; Demo Mode ($08)
ptr_GM_Level: bra.w  GM_Level     ; Normal Level ($0C)
ptr_GM_Special: bra.w GM_Special   ; Special Stage ($10)
ptr_GM_Cont:  bra.w  GM_Continue   ; Continue Screen ($14)
ptr_GM_Ending: bra.w GM_Ending     ; End of game sequence ($18)
ptr_GM_Credits: bra.w GM_Credits   ; Credits ($1C)

    rts
; =====

ChecksumError:
    bsr.w  VDPSetupGame
    move.l #$C0000000,(vdp_control_port).l ; set VDP to CRAM write
    moveq  #($80)/2-1,d7

```

Locate MainGameLoop: in the .ASM file.

Change the **Game Mode Limit** value from **\$1C** to **\$7C** (this will be helpful if you wish to add more splash screens of this type.)

```
andi.w #$1C,d0 ; limit Game Mode value to $1C max (change to a maximum of 7C to add more game modes)
```

Locate this line of code.

```
andi.w #$7C,d0 ; limit Game Mode value to $1C max (change to a maximum of 7C to add more game modes)
```

Change the Game Mode Limit value to \$7C.

Locate the **game mode array**. This should be located right below the **MainGameLoop** line.

```

GameModeArray:
ptr_GM_Sega:  bra.w  GM_Sega      ; Sega Screen ($00)
ptr_GM_Title: bra.w  GM_Title     ; Title Screen ($04)
ptr_GM_Demo:  bra.w  GM_Level     ; Demo Mode ($08)
ptr_GM_Level: bra.w  GM_Level     ; Normal Level ($0C)
ptr_GM_Special: bra.w GM_Special   ; Special Stage ($10)
ptr_GM_Cont:  bra.w  GM_Continue   ; Continue Screen ($14)
ptr_GM_Ending: bra.w GM_Ending     ; End of game sequence ($18)
ptr_GM_Credits: bra.w GM_Credits   ; Credits ($1C)

    rts

```

Locate the game mode array.

After “(\$1C)” but before “rts,” press the **ENTER** key **twice** to go down by two spaces, then **copy the code below**:

```
ptr_GM_SplashTest: bra.w GM_SplashTest    ; SplashTest ($20)
```

Copy the code above. You can change “SplashTest” in the text to the name of the files you saved (if you created your own file names).

```
ptr_GM_Credits: bra.w GM_Credits    ; Credits ($1C)

ptr_GM_SplashTest:    bra.w GM_SplashTest    ; SplashTest ($20)|

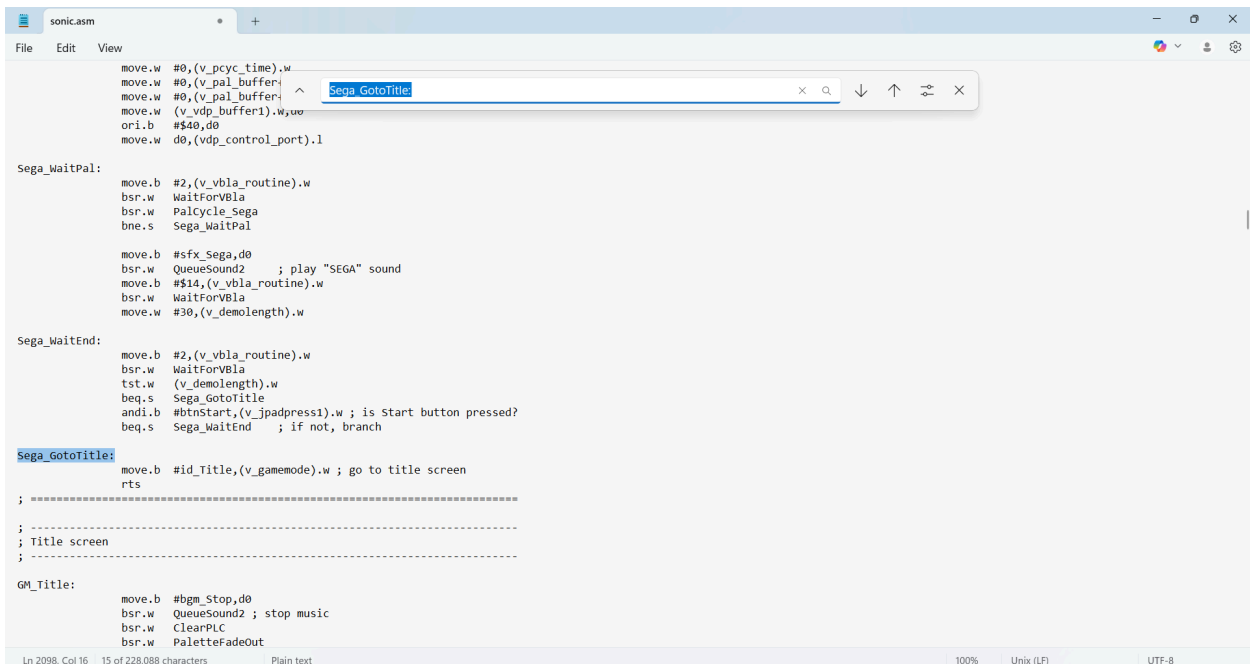
                        rts
```

Paste the code above in Notepad.

Now comes the **final addition** to the **sonic.asm** file: **the game mode itself**.

For this tutorial, we’ll be placing the **splash screen in-between** the **Sega screen** and the “**Sonic Team Presents**”/title screen. (If you wish, you can add more splash screens or move the existing splash screen(s) to another portion of the game. Be aware of long-distance jump errors.)

Locate **Sega_GotoTitle**: in the file (**CTRL+F** for faster searching).

A screenshot of a text editor window titled 'sonic.asm'. A search bar at the top right shows 'Sega_GotoTitle' with a magnifying glass icon. The code is in assembly language. The search results highlight the 'Sega_GotoTitle' label. The code includes sections for Sega title screen, GM title screen, and various routines like Sega_WaitPal and Sega_WaitEnd. The status bar at the bottom shows 'Ln 2098, Col 16', '15 of 228,088 characters', 'Plain text', '100%', 'Unix (LF)', and 'UTF-8'.

Locate **Sega_GotoTitle**: in the **.ASM** file.

Change the **game mode ID** from “Title” to “**SplashTest.**” (If you named the game mode as something **other than “SplashTest,”** change the **game mode ID to that name** instead.)

```
move.b #id_Title,(v_gamemode).w ; go to title screen
```

```
move.b #id_SplashTest,(v_gamemode).w ; go to title screen
```

Change the game mode ID from “Title” to “SplashTest.”

Locate the **blank space** between the end of the **Sega** screen game mode and the **Title** screen game mode and press the **ENTER** key **twice** to go down by two spaces. **Move up one line.**

```
Sega_GotoTitle:
```

```
    move.b #id_SplashTest,(v_gamemode).w ; go to title screen
    rts
```

```
; =====
; -----
; Title screen
; -----
```

Locate the blank space between the end of the Sega screen game mode and the Title screen game mode.

```
Sega_GotoTitle:
```

```
    move.b #id_SplashTest,(v_gamemode).w ; go to title screen
    rts
```

```
; =====
; -----
; Title screen
; -----
```

Press the enter key twice and then move up one line by using the up arrow key.

Copy all of the code below:

```
;-----  
; TEST SPLASH SCREEN  
;-----  
GM_SplashTest:  
    move.b    #bgm_Fade,d0  
    bsr.w    ClearPLC  
    bsr.w    PaletteFadeOut  
    lea      (vdp_control_port).l,a6  
    move.w    #$8004,(a6) ; use 8-colour mode  
    move.w    #$8200+(vram_fg>>10),(a6) ; set foreground  
nametable address  
    move.w    #$8400+(vram_bg>>13),(a6) ; set background  
nametable address  
    move.w    #$9001,(a6)      ; 64-cell hscroll size  
    move.w    #$9200,(a6)      ; window vertical position  
    move.w    #$8B03,(a6)      ; line scroll mode  
  
    clr.b    (f_wtr_state).w  
    bsr.w    ClearScreen  
  
    lea      (v_objspace).w,a1  
    moveq     #0,d0  
    move.w    #$7FF,d1  
  
GM_SplashTest_ClrObjRam:  
    move.l    d0,(a1)+  
    dbf      d1,GM_SplashTest_ClrObjRam ; clear object RAM  
  
    lea      (v_palette_fading).w,a1  
    moveq     #cBlack,d0  
    move.w    #$1F,d1  
  
.blackfill0:  
    move.l    d0,(a1)+
```

```
dbf    d1,.blackfill0 ; fill palette with black
```

```
move.l #$40000000,($C00004).l
```

```
lea    (Nem_SplashTest).l,a0
```

```
bsr.w  NemDec
```

```
lea    ($FF0000).l,a1
```

```
lea    (Eni_SplashTest).l,a0 ; load mappings for Background Art
```

```
move.w    #0,d0
```

```
bsr.w  EniDec
```

```
copyTilemap v_256x256&FFFFFF,vram_fg,40,28
```

```
moveq    #palid_SplashTest,d0
```

```
bsr.w  PalLoad_Fade      ; load Sonic's palette
```

```
clr.w  (v_palette_fading+$40).w
```

```
jsr    (ExecuteObjects).l
```

```
jsr    (BuildSprites).l
```

```
bsr.w  PaletteFadeIn
```

GM_SplashTest_MainLoop:

```
move.b    #4,(v_vbla_routine).w
```

```
bsr.w  WaitForVBla
```

```
jsr    (ExecuteObjects).l
```

```
jsr    (BuildSprites).l
```

```
andi.b #btnStart,(v_jpadpress1).w
```

```
beq.s  GM_SplashTest_MainLoop
```

```
move.w    #$638,(v_demolength).w ; run splash screen for
```

\$638 frames

move.b #04,(\$FFFFF600).w ; go to Title Screen

GM_SplashTest_Return:

rts

Copy the code above. You can change “SplashTest” in the code to the name of the files you saved (if you created your own file names) and/or the name of the game mode you named. “TEST SPLASH SCREEN” can also be renamed to whatever you’d like.

***Tips/Notice:** if you plan on having **more than one splash screen** in your hack, make sure to **rename the .blackfill0** portion of the code to the number following the previous number (ex. **.blackfill0, .blackfill1, .blackfill2, etc.**). If you want this splash screen to transition to another game mode (or another splash screen), go to **move.b #04,(\$FFFFF600).w ; go to Title Screen** in the code and change the game mode value from **#04 (the title screen)** to another game mode value (values go in a specific order, ex. **\$00, \$04, \$08, \$10, \$14, \$18, \$20, etc.**).*

*Also, you can select between **two** options on how you would like the **previous music track** to **stop**; by default, the **BGM** is set to “**#bgm_Fade**” which will **fade out the previous track**. Changing this to “**#bgm_Stop**” will make the **track abruptly stop** instead.*

```
Sega_GotoTitle:
    move.b #id_SplashTest,(v_gamemode).w ; go to title screen
    rts
; =====
; TEST SPLASH SCREEN
; =====
GM_SplashTest:
    move.b #bgm_Fade,d0
    bsr.w ClearPAC
    bsr.w PaletteFadeOut
    lea (vdp_control_port).l,a0
    move.w #0004,(a0) ; use 8-colour mode
    move.w #0100+(vram_fg>>10),(a0) ; set foreground nametable address
    move.w #0000+(vram_bg>>10),(a0) ; set background nametable address
    move.w #0001,(a0) ; 64-cell hscroll size
    move.w #0000,(a0) ; window vertical position
    move.w #0000,(a0) ; line scroll mode

    clr.b (f_vtr_state).w
    bsr.w ClearScreen

    lea (v_objspace).w,a1
    moveq #0,d0
    move.w #0000,d1

GM_SplashTest_ClearObjRam:
    move.l d0,(a1)+
    dbf d1,GM_SplashTest_ClearObjRam ; clear object RAM

    lea (v_palette_fading).w,a1
    moveq #black,d0
    move.w #0000,d1

.blackfill0:
    move.l d0,(a1)+
    dbf d1,.blackfill0 ; fill palette with black

    move.l #00000000,(SCREEN00).l
    lea (Mem_SplashTest).l,a0
    bsr.w MemDec

    lea ($FFFF0000).l,a1
    lea (Eni_SplashTest).l,a0 ; load mappings for Background Art

    move.w #0,d0
    bsr.w EndDec

    copyTilemap v_256x256$FFFFFF,vram_fg,a0,20

    moveq #id_SplashTest,d0
    bsr.w PalLoad_Fade ; load Sonic's palette
    clr.w (v_palette_fading$40).w
    jsr (ExecuteObjects).l
    jsr (BuildSprites).l
    bsr.w PaletteFadeIn

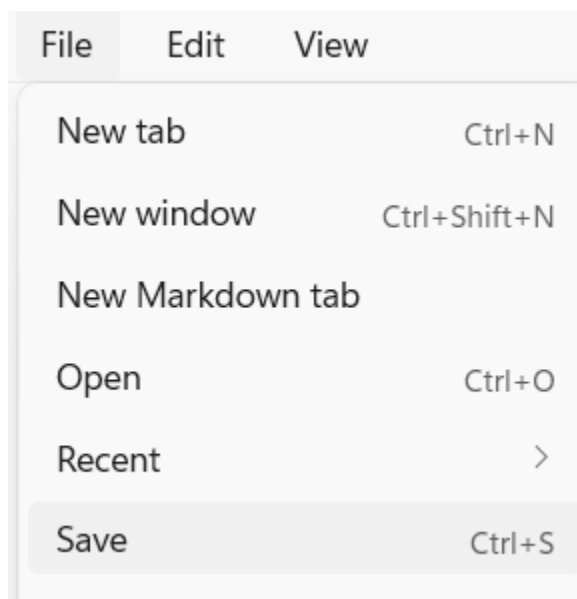
GM_SplashTest_MainLoop:
    move.b #0,(v_obja_routine).w
    bsr.w WaitForVBL
    jsr (ExecuteObjects).l
    jsr (BuildSprites).l
    andi.b #btnStart,(v_jpadpress).w
    beq.s GM_SplashTest_MainLoop
    move.w #000,(v_demoLength).w ; run splash screen for 500 frames

    move.b #04,($FFFFF600).w ; go to Title Screen

GM_SplashTest_Return:
    rts
; =====
; Title screen
; =====
```

Paste the code above in Notepad.

This is all you need to change in the **sonic.asm** file. Go to **File -> Save** and **close Notepad**.
(**WARNING: Make sure that you have backed up your disassembly prior to saving this file.**
Keep this in mind before saving the other .ASM files.)



Go to File -> Save and then close Notepad.

Next, open up **Constants.asm** in **Notepad**. It should also be found in the **root** directory of your **disassembly**.

	chkbitperfect.lua	9/1/2025 8:40 PM	LUA File	2 KB
	Constants.asm	9/1/2025 8:40 PM	ASM File	18 KB
	Macros.asm	9/1/2025 8:40 PM	ASM File	8 KB

Open Constants.asm in Notepad.

You should see this upon opening the file:

```

; Constants
; -----
Size_of_SegaPCM: equ $6078
Size_of_DMC_driver_guess: equ $1760

; Clocks
Master_Clock: equ $3691375
M68000_Clock: equ Master_Clock/7
Z80_Clock: equ Master_Clock/15
FM_Sample_Rate: equ M68000_Clock/(6*6*4)
PSG_Sample_Rate: equ Z80_Clock/16

; VDP addresses
vdp_data_port: equ $C00000
vdp_control_port: equ $C00004
vdp_counter: equ $C00008
pgm_input: equ $C00011

; Z80 addresses
z80_ram: equ $A00000 ; start of Z80 RAM
z80_ram_end: equ $A02000 ; end of non-reserved Z80 RAM
z80_version: equ $A10001
z80_port_1_data: equ $A10002
z80_port_1_control: equ $A10008
z80_port_2_control: equ $A1000A
z80_expansion_control: equ $A1000C
z80_bus_request: equ $A11200
z80_reset: equ $A11200
ym2612_a0: equ $A00000
ym2612_0b: equ $A00001
ym2612_a1: equ $A04002
ym2612_d1: equ $A04003

sram_port: equ $A130F1
security_addr: equ $A14000

```

Locate **; Game modes** in the file (CTRL+F for faster searching).

```

; VRAM data
vram_fg: equ $C000 ; foreground
vram_bg: equ $E000 ; background
vram_sprites: equ $F800 ; sprite table
vram_hscroll: equ $FC00 ; horizontal scroll table
tile_size: equ 8*8/2
plane_size_64x32: equ 64*32*2

; Game modes
id_Sega: equ ptr_GM_Sega-GameModeArray ; $00
id_Title: equ ptr_GM_Title-GameModeArray ; $04
id_Demo: equ ptr_GM_Demo-GameModeArray ; $08
id_Level: equ ptr_GM_Level-GameModeArray ; $0C
id_Special: equ ptr_GM_Special-GameModeArray ; $10
id_Continue: equ ptr_GM_Cont-GameModeArray ; $14
id_Ending: equ ptr_GM_Ending-GameModeArray ; $18
id_Credits: equ ptr_GM_Credits-GameModeArray ; $1C

```

Locate **; Game modes** in the .ASM file.

After “\$1C,” press the **ENTER** key to go down by one space, then **copy the code below**:

id_SplashTest: equ ptr_GM_SplashTest-GameModeArray; \$20

Copy the code above. You can change “SplashTest” in the text to your game mode name (if you decided to name your game mode.)

Tip: note that the **order of the game mode values is different** compared to how the **splash screen code interprets them (\$00, \$04, \$08, \$0C, \$10, \$14, \$18, \$1C, \$20, etc. instead of \$00, \$04, \$08, \$10, \$14, \$18, \$20, etc.).**

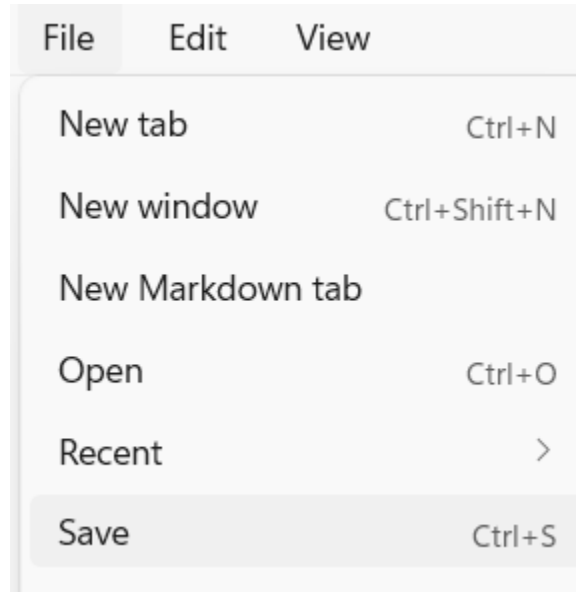
```

; Game modes
id_Sega:      equ ptr_GM_Sega-GameModeArray    ; $00
id_Title:    equ ptr_GM_Title-GameModeArray    ; $04
id_Demo:     equ ptr_GM_Demo-GameModeArray     ; $08
id_Level:    equ ptr_GM_Level-GameModeArray    ; $0C
id_Special:  equ ptr_GM_Special-GameModeArray  ; $10
id_Continue: equ ptr_GM_Cont-GameModeArray     ; $14
id_Ending:   equ ptr_GM_Ending-GameModeArray   ; $18
id_Credits:  equ ptr_GM_Credits-GameModeArray  ; $1C
id_SplashTest: equ ptr_GM_SplashTest-GameModeArray; $20

```

Paste the code above in Notepad.

This is all you need to change in the **Constants.asm** file. Go to **File -> Save** and **close Notepad**.



Go to File -> Save and then close Notepad.

Next, open up **Palette Pointers.asm** in **Notepad**. It should be found in the **_inc** folder in your **disassembly**.

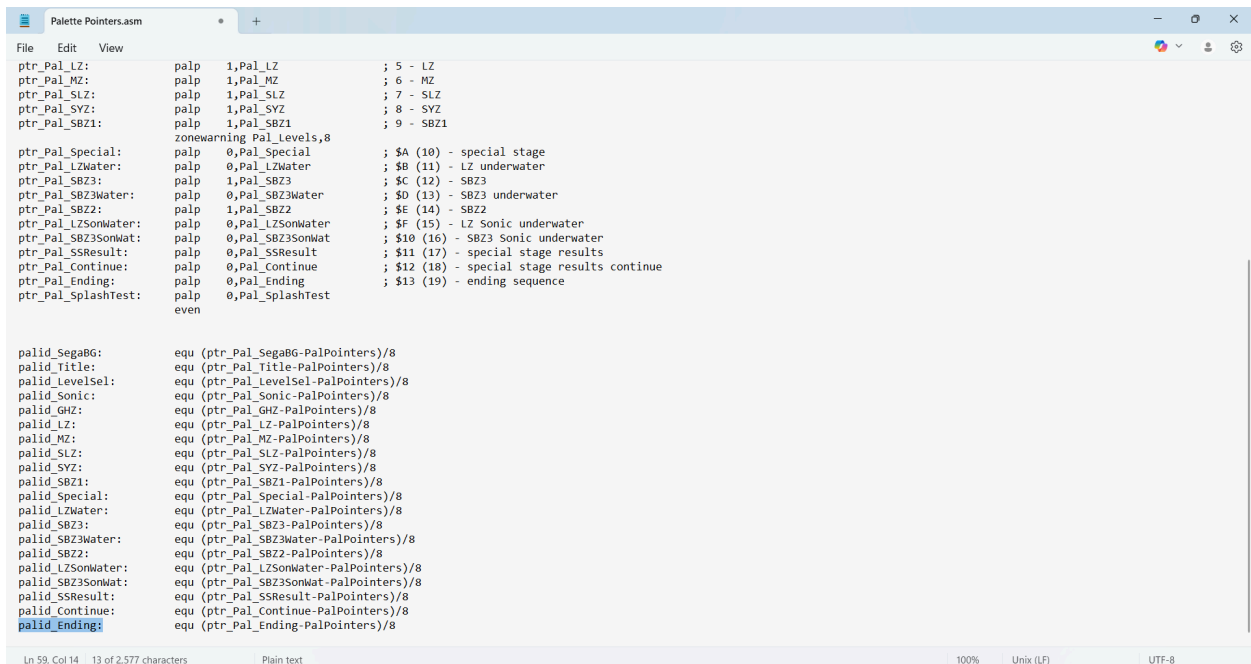
After “ending sequence” but before “even,” press the **ENTER** key to go down by one space, then **copy the code below**:

```
ptr_Pal_SplashTest: palp 0, Pal_SplashTest
```

Copy the code above. You can change “SplashTest” in the text to the name of the files you saved (if you created your own file names).

Tip/Notice: you can repeat this step for another splash screen. *The 0 always stays a 0.*

Now, locate **palid_Ending:** in the file by **scrolling to the bottom of the file**.



```
File Edit View
ptr_Pal_LZ:      palp 1, Pal_LZ      ; 5 - LZ
ptr_Pal_MZ:      palp 1, Pal_MZ      ; 6 - MZ
ptr_Pal_SLZ:     palp 1, Pal_SLZ     ; 7 - SLZ
ptr_Pal_SVZ:     palp 1, Pal_SVZ     ; 8 - SVZ
ptr_Pal_SBZ1:    palp 1, Pal_SBZ1    ; 9 - SBZ1
ptr_Pal_Special: zonewarning Pal_Levels,8
ptr_Pal_LZWater: palp 0, Pal_Special ; $A (10) - special stage
ptr_Pal_SBZ3:    palp 0, Pal_LZWater ; $B (11) - LZ underwater
ptr_Pal_SBZ3Water: palp 1, Pal_SBZ3 ; $C (12) - SBZ3
ptr_Pal_SBZ2:    palp 0, Pal_SBZ3Water ; $D (13) - SBZ3 underwater
ptr_Pal_SBZ2:    palp 1, Pal_SBZ2 ; $E (14) - SBZ2
ptr_Pal_LZSonWater: palp 0, Pal_SBZ2 ; $F (15) - LZ Sonic underwater
ptr_Pal_SBZ3SonWat: palp 0, Pal_LZSonWater ; $10 (16) - SBZ3 Sonic underwater
ptr_Pal_SSResult: palp 0, Pal_SBZ3SonWat ; $11 (17) - special stage results
ptr_Pal_Continue: palp 0, Pal_SSResult ; $12 (18) - special stage results continue
ptr_Pal_Ending:  palp 0, Pal_Continue ; $13 (19) - ending sequence
ptr_Pal_SplashTest: palp 0, Pal_Ending
even

palid_SegaBG:    equ (ptr_Pal_SegaBG-PalPointers)/8
palid_Title:    equ (ptr_Pal_Title-PalPointers)/8
palid_LevelSel: equ (ptr_Pal_LevelSel-PalPointers)/8
palid_Sonic:    equ (ptr_Pal_Sonic-PalPointers)/8
palid_GHZ:      equ (ptr_Pal_GHZ-PalPointers)/8
palid_LZ:       equ (ptr_Pal_LZ-PalPointers)/8
palid_MZ:       equ (ptr_Pal_MZ-PalPointers)/8
palid_SLZ:      equ (ptr_Pal_SLZ-PalPointers)/8
palid_SVZ:      equ (ptr_Pal_SVZ-PalPointers)/8
palid_SBZ1:     equ (ptr_Pal_SBZ1-PalPointers)/8
palid_Special:  equ (ptr_Pal_Special-PalPointers)/8
palid_LZWater:  equ (ptr_Pal_LZWater-PalPointers)/8
palid_SBZ3:     equ (ptr_Pal_SBZ3-PalPointers)/8
palid_SBZ3Water: equ (ptr_Pal_SBZ3Water-PalPointers)/8
palid_SBZ2:     equ (ptr_Pal_SBZ2-PalPointers)/8
palid_LZSonWater: equ (ptr_Pal_LZSonWater-PalPointers)/8
palid_SBZ3SonWat: equ (ptr_Pal_SBZ3SonWat-PalPointers)/8
palid_SSResult: equ (ptr_Pal_SSResult-PalPointers)/8
palid_Continue: equ (ptr_Pal_Continue-PalPointers)/8
palid_Ending:   equ (ptr_Pal_Ending-PalPointers)/8
```

Locate **palid_Ending:** in the .ASM file.

After “/8,” press the **ENTER** key to go down by one space, then **copy the code below**:

```
palid_SplashTest: equ (ptr_Pal_SplashTest-PalPointers)/8
```

Copy the code above. You can change “SplashTest” in the text to the name of the files you saved (if you created your own file names).

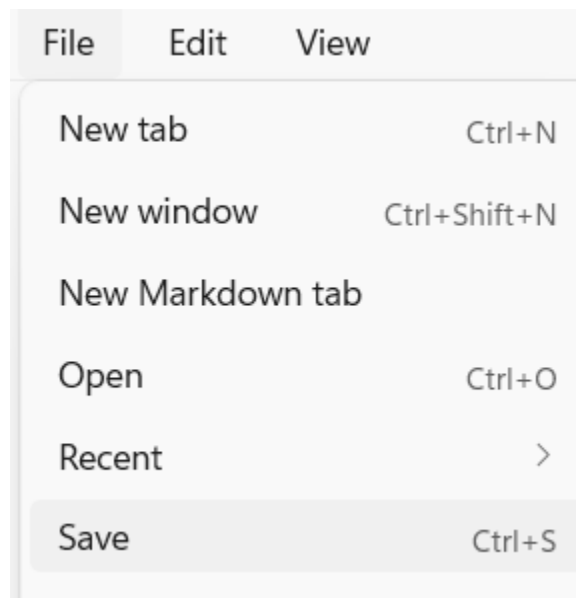
```

palid_SBZ3:      equ (ptr_Pal_SBZ3-PalPointers)/8
palid_SBZ3Water: equ (ptr_Pal_SBZ3Water-PalPointers)/8
palid_SBZ2:      equ (ptr_Pal_SBZ2-PalPointers)/8
palid_LZSonWater: equ (ptr_Pal_LZSonWater-PalPointers)/8
palid_SBZ3SonWat: equ (ptr_Pal_SBZ3SonWat-PalPointers)/8
palid_SSResult:  equ (ptr_Pal_SSResult-PalPointers)/8
palid_Continue:  equ (ptr_Pal_Continue-PalPointers)/8
palid_Ending:    equ (ptr_Pal_Ending-PalPointers)/8
palid_SplashTest: equ (ptr_Pal_SplashTest-PalPointers)/8

```

Paste the code above in Notepad.

This is all you need to change in the **Constants.asm** file. Go to **File -> Save** and **close Notepad**.



Go to File -> Save and then close Notepad.

Double-check all of your .ASM files. Once done, you may now proceed to the final section,
Building the Final Build: Splash Screen Implemented?

-BUILDING THE FINAL BUILD: SPLASH SCREEN IMPLEMENTED?-

Now, for the moment of truth: Did you successfully implement the splash screen?

Run build.bat and open up the created build.

 .gitignore	9/1/2025 8:40 PM	GITIGNORE File	1 KB
☒  build.bat	9/1/2025 8:40 PM	Windows Batch File	1 KB
 build.lua	9/1/2025 8:40 PM	LUA File	2 KB

Run build.bat, this batch file should be found in the root of your disassembly.

 sonic.lst	9/2/2025 12:49 PM	LST File	7,971 KB
☒  s1built.bin	9/2/2025 12:49 PM	BIN File	512 KB

Open the generated build in a Sega Genesis/Mega Drive emulator.

If you received an **error message** similar to this during the building process, **double-check the code, the error message will show where the problem is located** (and save a copy of the error in **sonic.log**, found in the **root** directory of your **disassembly**.)

```
C:\windows\system32\cmd.exe x + v
Uncompressed driver size: 1BC6h bytes.
> > > sonic.asm(2151):10: error #1010: symbol undefined
> > > palid_Splash
> > > moveq #palid_Splash,d0
> > >
*****
*                                     *
*      There were build errors. See sonic.log for more details.      *
*                                     *
*****
Press any key to continue . . . |
```

If you somehow mess something up in one of the .ASM files, your build will not generate and a message like this will display.

Otherwise, if your **build works** and the graphic shows up **after** the **Sega** screen, then congratulations, you've successfully implemented your very first splash screen to Sonic 1!



If you did everything correctly, this or your graphic should appear after the Sega screen. (Note the palette cycling at the bottom left, this confirms the legitimacy of the screenshot.)

Here is the build containing the tutorial splash screen graphic to show its legitimacy:

[The ROM hack \(compressed into a .ZIP file\)](#)

Note: this file may be blocked on certain browsers due to false malware claims, there is no virus or malware included in the file.

Feel free to try out some **extra material** to spice up your **splash screen!**

Thank you for using this guide to aid in your ROM-hacking endeavors!

-DOCUMENT INFO, CREDITS, & NOTES-

This document is written by TheGamer2000.

ROM hack testing by TheGamer2000.

Graphics by TheGamer2000.

Sonic the Hedgehog (character) by Sega.

Partial code derived from the Needlemouse: Return to Tokyo Final Build Project ROM hack.

Credits to ComputerNerd, MainMemory, and SonicRetro for providing public accessibility to the various programs and tools featured in this tutorial.

Note that the File Garden-based links may fail to work depending on the site's uptime.

The GitHub disassembly used in this tutorial was last updated on 8/30/2025. Older and future versions of this disassembly may require further tinkering with the code to support gradual changes or nonexistent changes.

This document is designed only for users running modern releases of Windows (10 to 11). If you are not a modern Windows user, you may need to find an alternative method to perform this task.

*Revision 01: Included a fix regarding a .7Z file extraction error found by Madness Prime.
(9/5/2025)*

©2025 TheGamer2000 and all aforementioned names, groups, and corporations.

This document is **REVISION #01, VERSION #2.**

**ROM HACKING
GUIDE**

